CS-253 Algorithm Design and Analysis Practical Project: Empirical Study of Sorting Algorithm Performance

Aryan Bokolia (241CS111)
Rishinandan D R (241CS145)

March 9, 2026

1 Introduction and Research Objectives

This investigation aims to conduct a comprehensive analysis of various sorting algorithms, focusing on the relationship between their theoretical computational complexity and actual observed performance characteristics. The study encompasses a range of fundamental sorting methods, including Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort (with different pivot selection techniques), Heap Sort, and Radix Sort.

This research prioritizes empirical performance evaluation over theoretical implementation details. Through systematic experimentation with diverse input datasets and comprehensive timing measurements across varying input sizes and initial configurations (randomized, pre-sorted, and reverse-ordered sequences), we seek to establish concrete insights regarding the practical efficiency and behavioral patterns of these algorithms in real computational environments.

2 Experimental Framework and Methodology

To establish measurement reliability and experimental reproducibility, the following testing infrastructure and procedural approach were implemented:

- **Computational Environment:** The experiments were carried out on an HP Victus laptop powered by an Intel Core i7 processor and NVIDIA RTX 4050 GPU. The implementation and testing were performed in a Kali Linux environment hosted on Oracle VirtualBox.
- **Timing Implementation:** The runtime of each algorithm was recorded within the C programs using the `clock_gettime` function with the `CLOCK_MONOTONIC` clock source, ensuring precise measurement independent of external execution overhead.
- **Experimental Repetition Protocol:** Each experimental configuration was executed a total of **7 independent runs**, and observations were aggregated to obtain consistent and reliable performance measurements.

- **Performance Reporting:** For each test case, the final timing value corresponds to the **average execution duration** computed over all runs, thereby smoothing out fluctuations caused by transient system conditions.
- **Dataset Selection Strategy:** The datasets were generated in advance using a dedicated script, producing three distinct categories: randomly ordered values, already sorted sequences, and reverse-ordered sequences. Multiple input sizes were considered to observe performance trends across varying scales.
- **Input Consistency Protocol:** Yes, identical datasets were supplied to every sorting algorithm to maintain fairness in comparison, ensuring that performance differences arise solely from the algorithmic behavior rather than variations in input.

3 Quick Sort Variant Performance Analysis

We evaluated three distinct Quick Sort implementations based on their pivot selection methodologies:

1. **Strategy 1 (First Element Pivot):** Employs the initial array element as the partitioning pivot.
2. **Strategy 2 (Random Element Pivot):** Utilizes a randomly selected array element as the pivot.
3. **Strategy 3 (Median-of-Three Pivot):** Selects the median value from the first, middle, and last array elements as the pivot.

3.1 Theoretical Performance Characteristics

Quick Sort efficiency is fundamentally dependent on pivot selection quality. The algorithmic complexity for different scenarios can be expressed through recurrence relations:

- **Optimal Case:** $T(n) = 2T(n/2) + O(n) \implies O(n \log n)$
- **Degraded Case:** $T(n) = T(n-1) + O(n) \implies O(n^2)$
- **Expected Case:** $T(n) = 2T(n/2) + O(n) \implies O(n \log n)$

3.2 Experimental Observations

The First Element Pivot strategy exhibits quadratic degradation ($O(n^2)$) when processing sorted or reverse-sorted input sequences. Our experimental measurements demonstrate that for datasets of $N = 100,000$ elements in reverse-sorted configuration, the First Pivot implementation required **9.993 seconds** for completion, whereas the Median-of-Three and Random Pivot variants completed in **0.0069 seconds** and **0.0064 seconds** respectively.

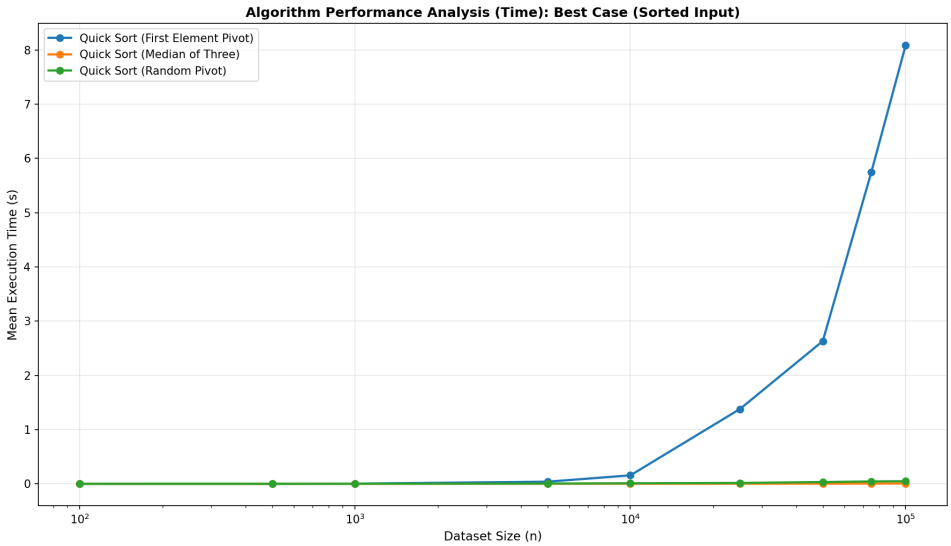

Figure 1: Quick Sort Variants: Best Case (Sorted Input) - Linear Scale

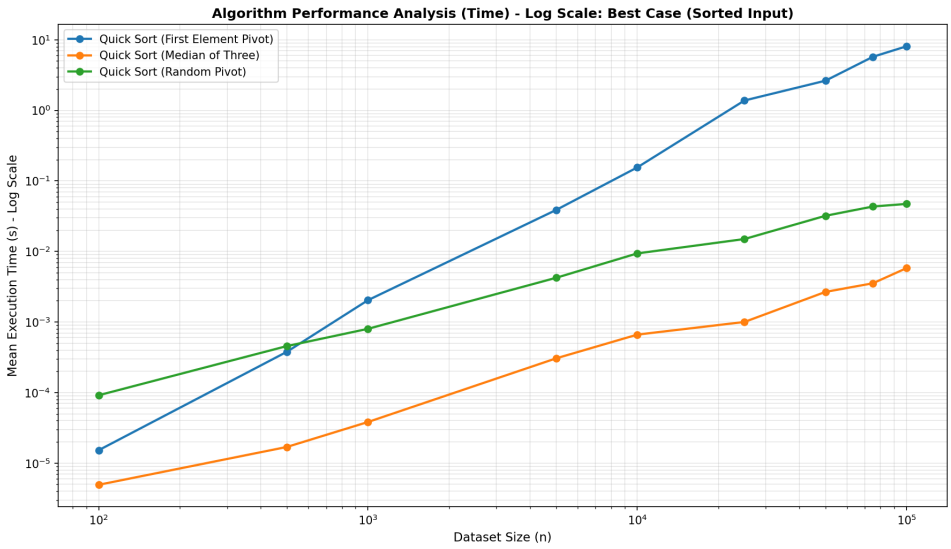

Figure 2: Quick Sort Variants: Best Case (Sorted Input) - Logarithmic Scale

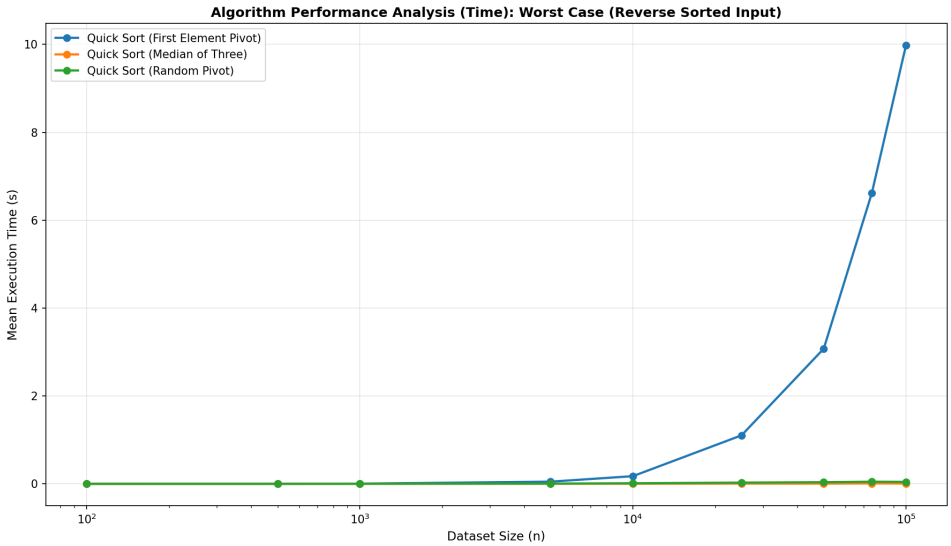

Figure 3: Quick Sort Variants: Worst Case (Reverse Sorted Input) - Linear Scale

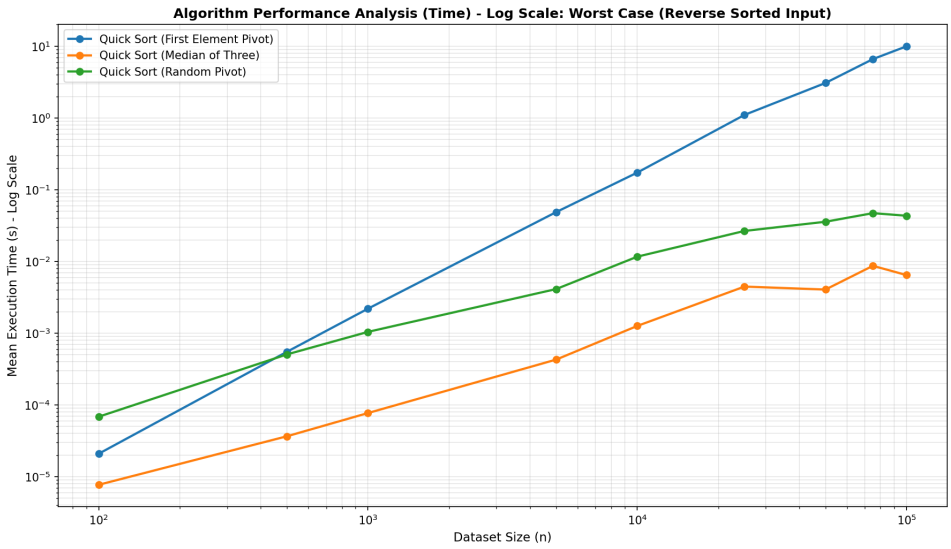

Figure 4: Quick Sort Variants: Worst Case (Reverse Sorted Input) - Logarithmic Scale

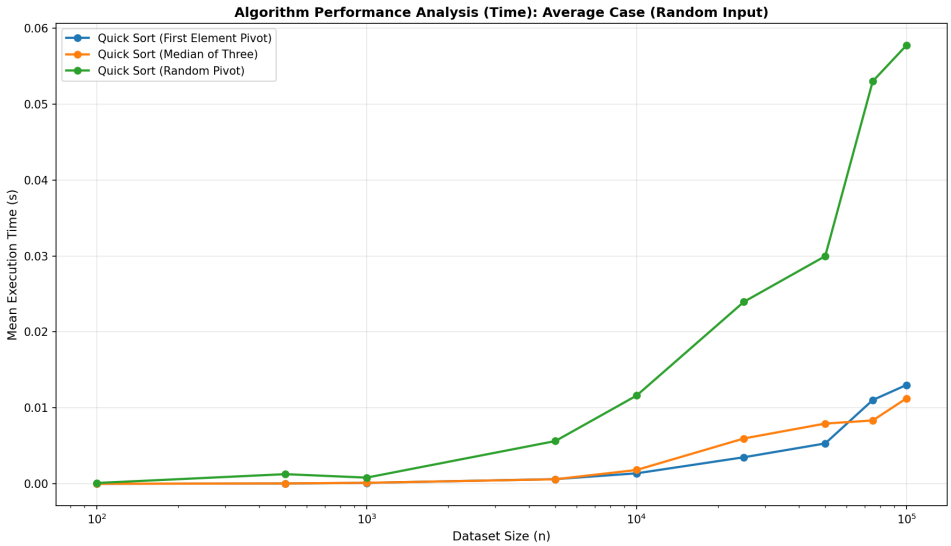

Figure 5: Quick Sort Variants: Average Case (Random Input) - Linear Scale

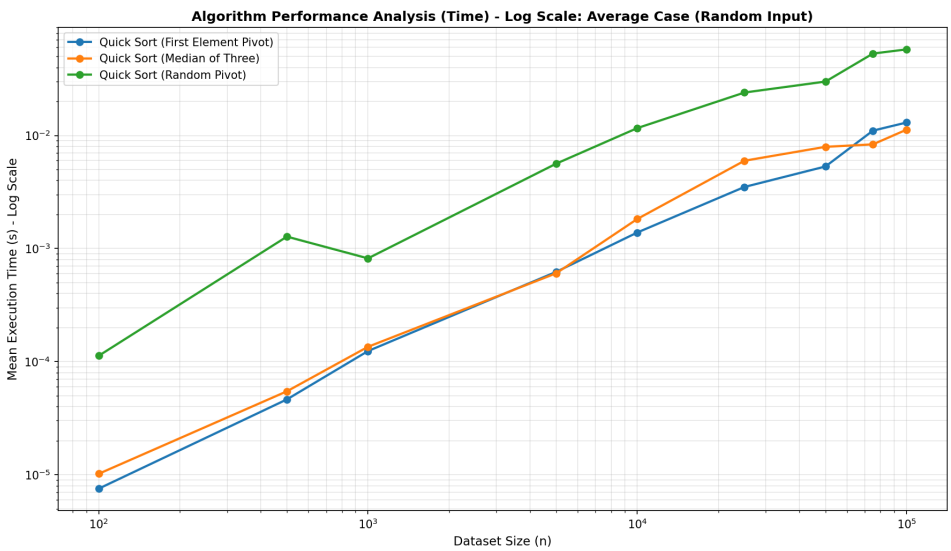

Figure 6: Quick Sort Variants: Average Case (Random Input) - Logarithmic Scale

3.3 Performance Assessment

The **Median-of-Three Quick Sort demonstrates superior overall performance** attributable to its enhanced pivot selection methodology. In conventional First Pivot Quick Sort implementations, utilizing the initial array element as pivot when processing already sorted or reverse-sorted datasets results in severely imbalanced partitioning (generating sub-arrays of dimensions 0 and $n-1$). This condition precipitates quadratic complexity ($O(n^2)$) and introduces potential stack overflow risks due to excessive recursion depth, as evidenced by our experimental results where the First Pivot variant required approximately 10 seconds for processing $N = 100,000$ elements.

The Median-of-Three approach addresses this limitation by selecting the median value from the first, middle, and terminal array elements. This heuristic substantially increases the probability of selecting a pivot value approximating the true dataset median, thereby

maintaining more equitable partitioning even with structured input configurations (sorted or nearly sorted data). By preserving partition ratios approaching $1 : 1$, the recurrence relationship remains $T(n) = 2T(n/2) + O(n)$, sustaining logarithmic complexity $O(n \log n)$. Although the Random Pivot strategy also circumvents quadratic degradation, the Median-of-Three technique typically demonstrates marginal performance advantages in practical applications by eliminating random number generation overhead while providing comparable protection against pathological input scenarios.

4 Comprehensive Algorithm Performance Evaluation

Our investigation encompassed seven distinct sorting algorithms: Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort (Median-of-Three variant), Heap Sort, and Radix Sort.

4.1 Algorithmic Complexity Analysis

The subsequent table presents a comprehensive overview of theoretical temporal and spatial complexities for the seven evaluated algorithms:

Table 1: Computational Complexity Characteristics of Sorting Algorithms

Algorithm	**Optimal Case**	**Expected Case**	**Degraded Case**	**Memory Requireme**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort (Med3)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$

4.2 Empirical Performance Assessment

When processing randomized datasets ($N = 100{,}000$ elements), **Radix Sort** demonstrated optimal performance with **0.0047 seconds** execution time, followed by **Quick Sort (Median-of-Three)** at **0.0097 seconds**. For pre-sorted input configurations, **Bubble Sort** and **Insertion Sort** exhibited exceptional performance (**0.0002 seconds**) due to their linear-time optimization capabilities for ordered sequences.

Conversely, **Selection Sort** demonstrated consistently inferior performance across all input configurations due to its inherent quadratic complexity characteristics, rendering it unsuitable for large-scale dataset processing. The experimental results further confirm that $O(n \log n)$ **complexity algorithms** such as Merge Sort and Heap Sort maintain consistent and predictable performance irrespective of input distribution patterns. Moreover, the investigations highlight the substantial performance differential between quadratic and logarithmic growth rates as input dimensions increase. Both **linear-scale and logarithmic-scale visualizations** were incorporated to effectively represent performance trends and growth characteristics, where logarithmic representations particularly emphasize algorithmic complexity distinctions. Collectively, the empirical results

unequivocally demonstrate the critical importance of algorithmic efficiency and input characteristics in determining practical computational performance.

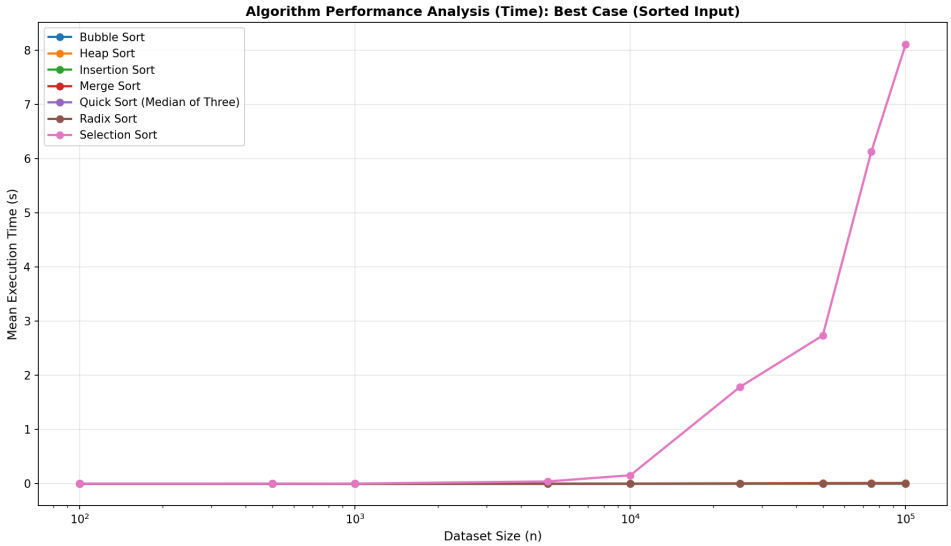

Figure 7: All Sorts: Best Case (Sorted Input) - Linear Scale

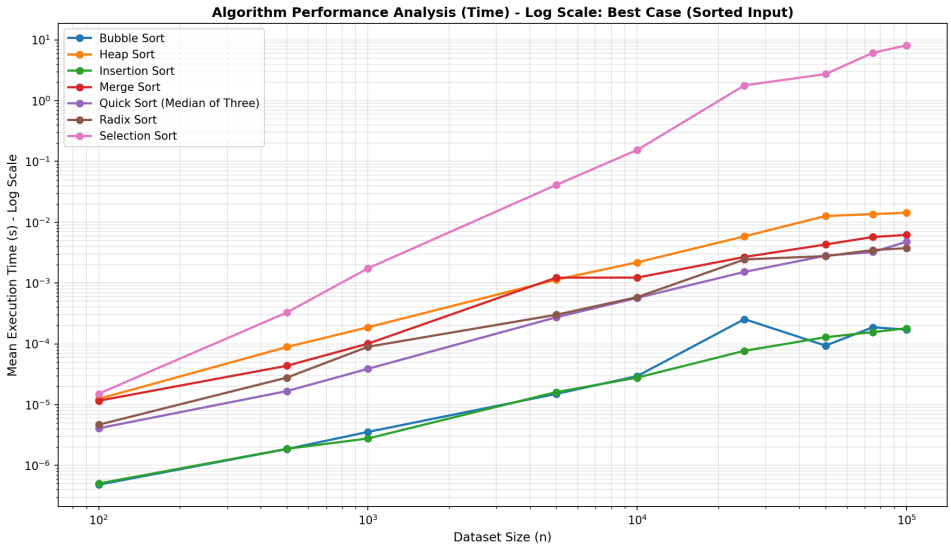

Figure 8: All Sorts: Best Case (Sorted Input) - Logarithmic Scale

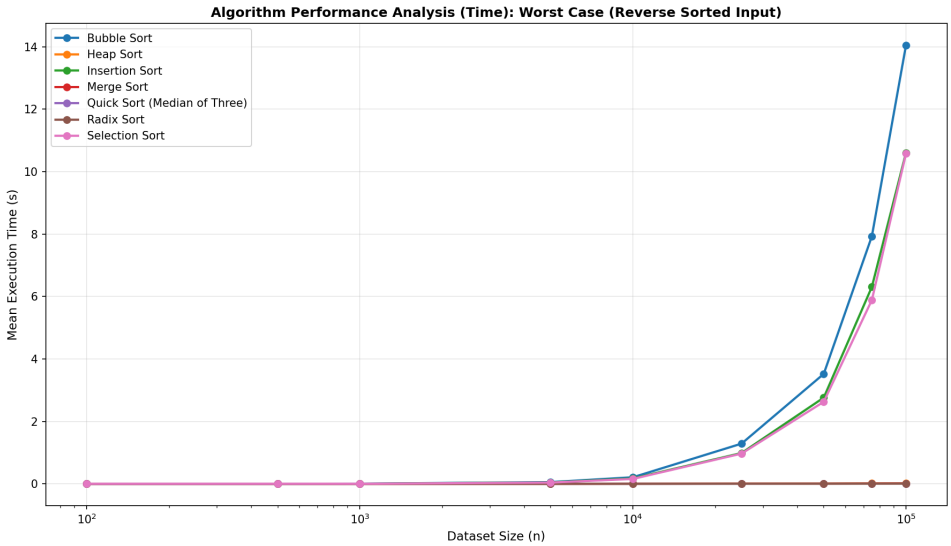

Figure 9: All Sorts: Worst Case (Reverse Sorted Input) - Linear Scale

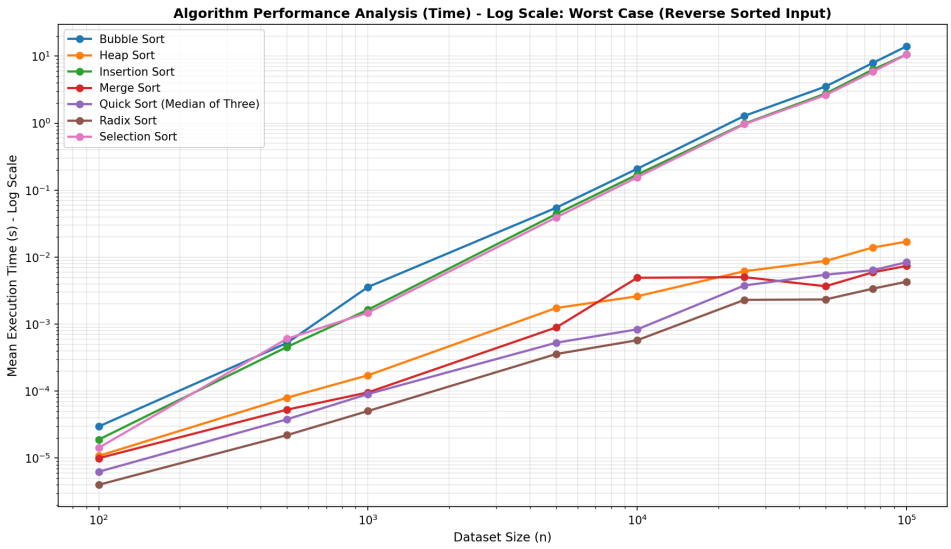

Figure 10: All Sorts: Worst Case (Reverse Sorted Input) - Logarithmic Scale

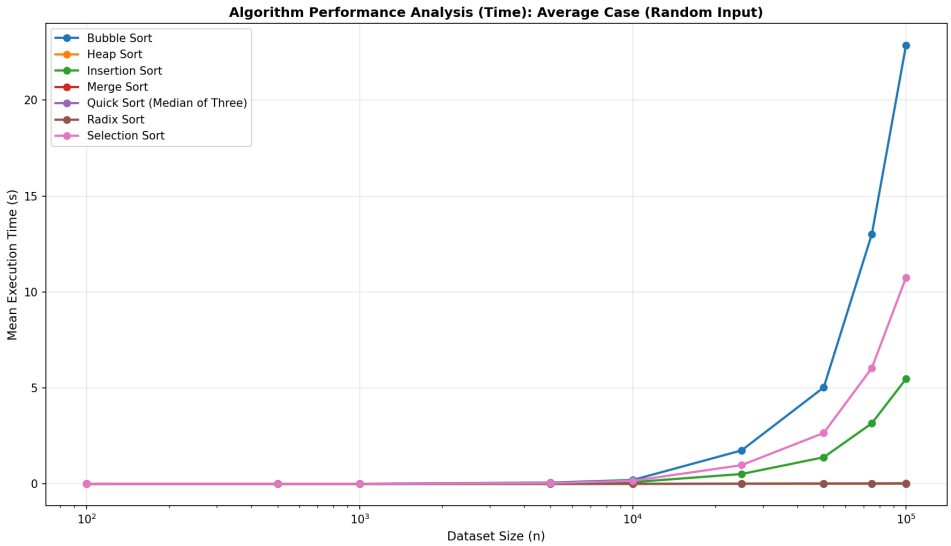

Figure 11: All Sorts: Average Case (Random Input) - Linear Scale

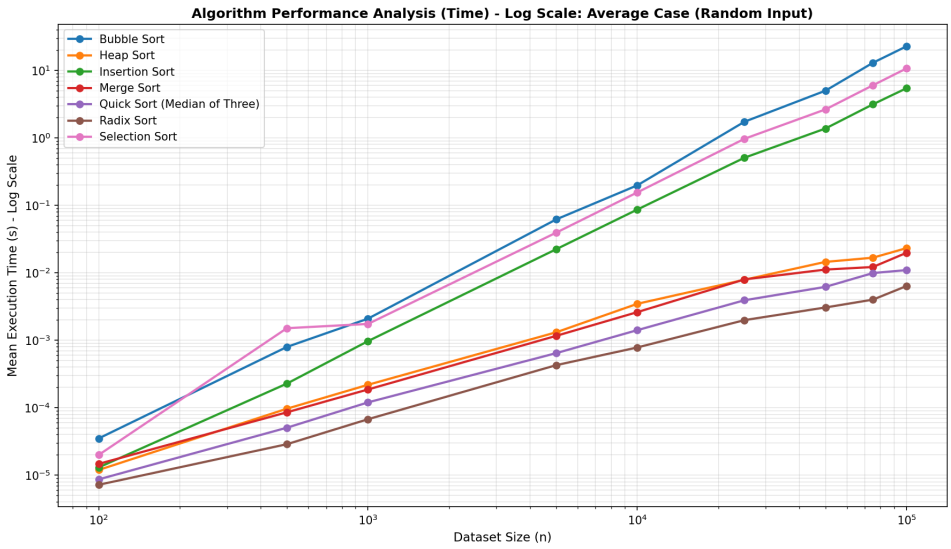

Figure 12: All Sorts: Average Case (Random Input) - Logarithmic Scale

4.3 Performance Conclusions

For extensive dataset processing, **Radix Sort** emerges as the predominant performer due to its linear-like computational characteristics. Among comparison-based sorting methodologies, **Quick Sort (Median-of-Three)** demonstrates optimal average-case efficiency, while **Merge Sort** and **Heap Sort** provide enhanced reliability for worst-case scenarios.

5 Statistical Relationship Between Comparison Operations and Execution Duration

The quantity of element comparisons serves as a primary determinant of computational duration for comparison-based sorting algorithms. Our statistical analysis confirms a robust positive correlation between these performance metrics.

Table 2: Statistical Correlation Analysis: Comparison Operations vs Execution Duration

Algorithm	**Correlation Coefficient (r)**	**Statistical Significance (p-value)**
Bubble Sort	0.9615	1.53×10^{-15}
Heap Sort	0.9816	1.68×10^{-19}
Insertion Sort	1.0000	2.94×10^{-53}
Merge Sort	0.9976	1.42×10^{-30}
Quick Sort (Med3)	0.8789	1.63×10^{-09}
Selection Sort	0.9996	8.77×10^{-40}

The near-perfect correlation observed for Insertion Sort ($r = 1.0$) and Selection Sort ($r = 0.9996$) indicates that their computational duration is predominantly governed by comparison operation count. Radix Sort, implementing a non-comparison-based methodology, exhibits a correlation of 0.9933 with its internal processing operations, though these do not represent traditional element-to-element comparisons.

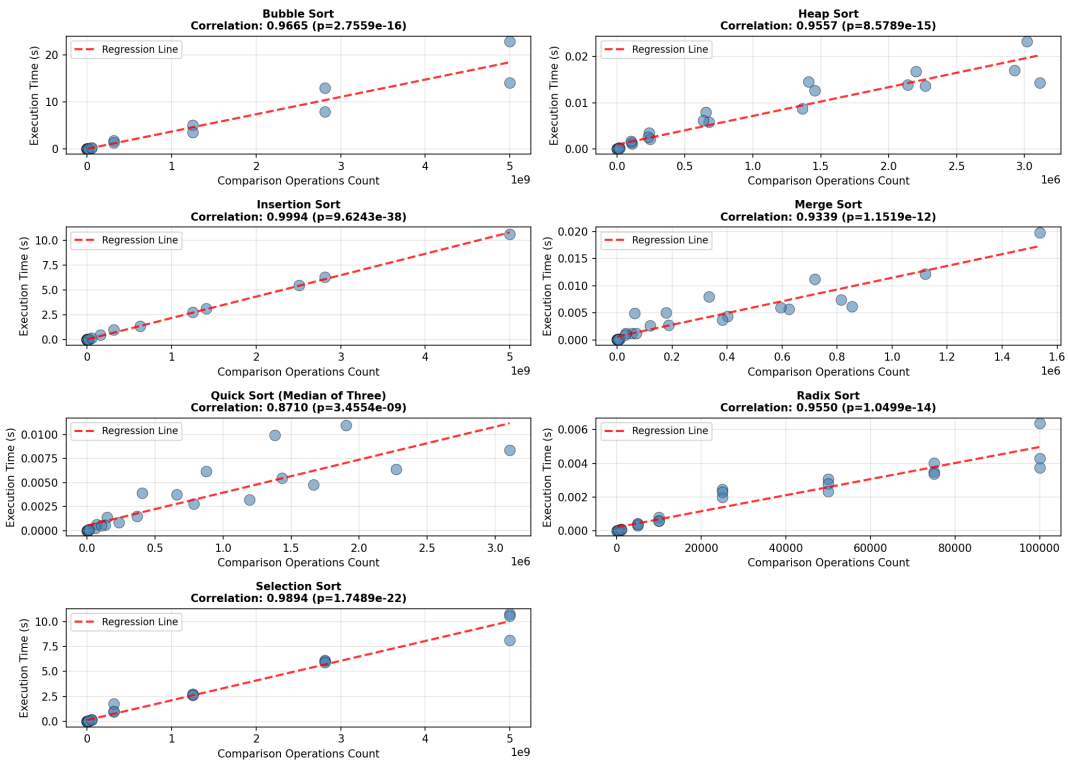

Figure 13: Statistical Correlation Analysis: Execution Duration vs Comparison Operation Count - Linear Scale

6 Research Conclusions and Implications

This investigation presents a comprehensive empirical evaluation of diverse sorting algorithms under varying input configurations, encompassing randomized, pre-sorted (optimal case), and reverse-ordered (degraded case) datasets. The experimental findings demonstrate substantial concordance between **empirical performance measurements and theoretical computational complexity predictions**, thereby validating the significance of asymptotic analysis in forecasting algorithmic behavior.

Algorithms exhibiting $O(n^2)$ **complexity characteristics**, including Bubble Sort, Selection Sort, and Insertion Sort (in average and worst-case scenarios), display accelerated growth in both execution duration and comparison operations as input dimensions expand. Despite their implementation simplicity and effectiveness for limited datasets, these algorithms become **computationally infeasible for large-scale inputs due to inherent scalability limitations**.

In contrast, $O(n \log n)$ **complexity algorithms** such as Merge Sort, Heap Sort, and Quick Sort manifest substantially enhanced scalability and performance characteristics for extensive dataset processing. The experimental results additionally emphasize that **pivot selection strategy in Quick Sort implementations is fundamentally crucial**, as naive methodologies demonstrate inferior performance with ordered inputs while median or randomized pivot approaches maintain computational efficiency.

The research further establishes that **input distribution patterns significantly influence algorithmic performance**. Adaptive algorithms including Bubble Sort and Insertion Sort achieve linear-time computational behavior for pre-sorted input sequences, whereas non-adaptive implementations like Selection Sort maintain quadratic complexity regardless of input ordering.

Furthermore, **non-comparison-based methodologies** such as Radix Sort demonstrate near-linear computational performance and surpass comparison-based algorithms under appropriate conditions.

Collectively, these findings confirm that **algorithm selection should be contingent upon input scale, data distribution characteristics, and application-specific requirements**, and that asymptotic complexity analysis remains a reliable predictor of practical performance despite implementation-specific factors and system-level overhead considerations.